

Laboratorio #2 – Esquemas de Detección y Corrección de Errores

Osman de León – 23428

Oliver Viau – 23544

Descripción de la Práctica

En esta práctica armamos un sistema de comunicación entre un cajero automático (que actúa como emisor) y un servidor bancario (el receptor). Para esto usamos una arquitectura por capas: Aplicación, Presentación, Enlace, Ruido y Transmisión. El emisor lo desarrollamos en Python y el receptor en Node.js, y la comunicación se hizo mediante sockets TCP. La capa de Presentación se encargó de pasar los mensajes a binario usando código ASCII. Después, en la capa de Enlace, agregamos información para verificar que los datos no estuvieran corruptos. Para eso probamos dos algoritmos: el Checksum de Fletcher, que solo detecta errores, y el Código de Hamming (SEC), que además de detectar, puede corregir errores de un solo bit.

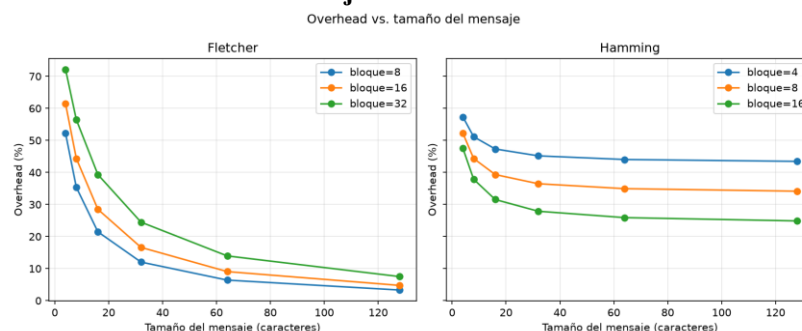
Antes de enviar la trama, la hicimos pasar por una capa de Ruido que simula un canal poco confiable, alterando bits al azar según una tasa de error que nosotros podíamos configurar. Del lado del receptor, la trama se recibía, se verificaba y, si usábamos Hamming, se corregía automáticamente siempre que el error estuviera dentro de un solo bit del bloque. Finalmente, el mensaje se decodificaba otra vez a texto y se mostraba en pantalla.

Además de la comunicación en sí, desarrollamos un conjunto de pruebas automáticas para evaluar cómo se comportaban ambos algoritmos con distintos tamaños de mensaje, distintos tamaños de bloque y diferentes probabilidades de error. Con esas pruebas calculamos métricas como el overhead, la tasa de detección y la tasa de corrección, que después usamos para analizar los resultados y comparar ambos métodos.

Resultados

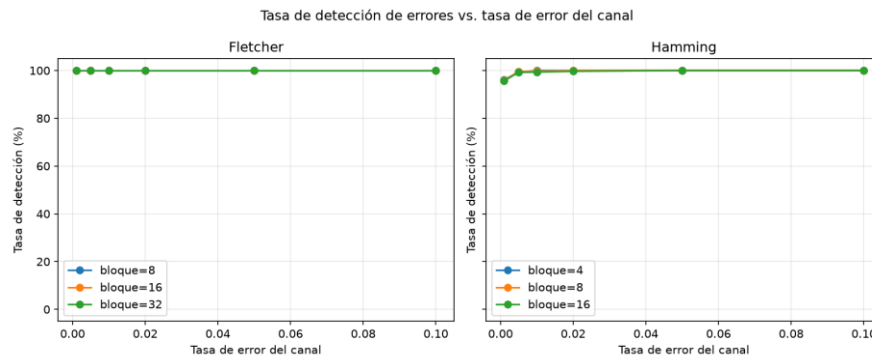
Para evaluar qué tan bien funcionaban los algoritmos, hicimos pruebas automáticas variando tres cosas: el tamaño del mensaje, el tamaño de bloque y la tasa de error del canal. Con los datos que obtuvimos, calculamos métricas clave como el porcentaje de overhead, la tasa de detección y la tasa de corrección. A continuación mostramos los resultados.

Figura 1. Overhead vs. tamaño del mensaje



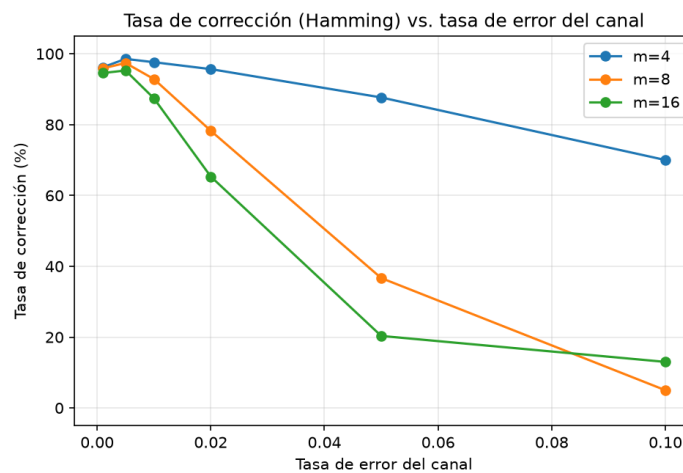
En la Figura 1 podemos ver cómo se comporta el overhead para Fletcher y Hamming al cambiar el tamaño del mensaje. En ambos casos, el overhead baja a medida que el mensaje es más largo, lo cual tiene sentido: la información de redundancia ocupa un porcentaje cada vez menor del total de la trama. También se nota que Fletcher genera menos sobrecarga que Hamming en casi todas las configuraciones que probamos.

Figura 2. Tasa de detección de errores vs. tasa de error del canal



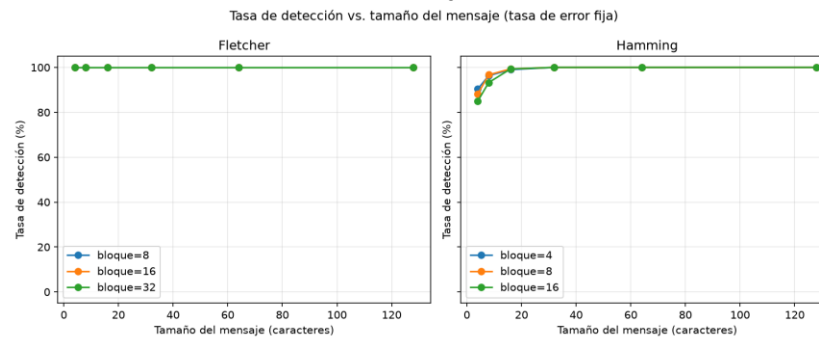
La Figura 2 muestra qué tan buenos son ambos algoritmos detectando errores cuando aumentamos la probabilidad de que el canal falle. En general, tanto Fletcher como Hamming mantienen tasas de detección muy altas, rozando el 100 %, y se mantienen estables en todas las configuraciones que analizamos.

Figura 3. Tasa de corrección de Hamming



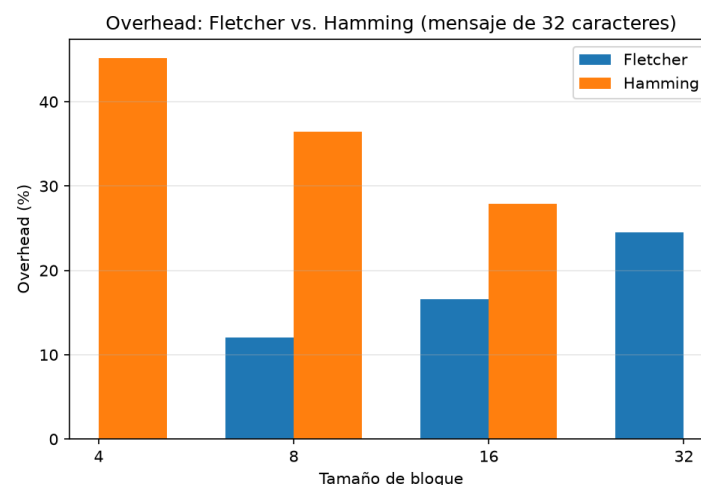
En la Figura 3 vemos la tasa de corrección del algoritmo de Hamming para distintos tamaños de bloque. Como era de esperar, la capacidad de corrección baja cuando la tasa de error del canal aumenta. Esto se debe a que, con más errores, es más probable que en un mismo bloque ocurra más de un bit erróneo, y Hamming solo puede corregir errores de un solo bit.

Figura 4. Tasa de detección vs. tamaño del mensaje



La Figura 4 compara la tasa de detección de errores al cambiar el tamaño del mensaje, manteniendo fija la tasa de error. Los resultados indican que el tamaño del mensaje no afecta prácticamente la capacidad de detección de ninguno de los dos algoritmos: en todas las pruebas se mantuvieron porcentajes igualmente altos.

Figura 5. Comparación del overhead entre Fletcher y Hamming



Por último, la Figura 5 compara directamente el overhead que generan Fletcher y Hamming para un mismo tamaño de mensaje. Se ve claramente que Fletcher necesita menos información extra para verificar la integridad de los datos. Hamming, en cambio, introduce más redundancia porque añade bits de paridad que le permiten corregir errores, y ese es el costo que hay que pagar por tener esa capacidad adicional.

Discusión

Los resultados que obtuvimos muestran que tanto Fletcher Checksum como el Código de Hamming cumplen bien con su función, aunque cada uno tiene sus fortalezas y limitaciones según las condiciones del canal.

En tema de overhead, Fletcher es más liviano: añade menos información extra que Hamming, sobre todo cuando los mensajes son largos. Esto es lógico porque Fletcher solo agrega un checksum al final de la trama, mientras que Hamming tiene que insertar bits de paridad en cada bloque para poder corregir errores. Así que, si lo que buscamos es ahorrar en datos transmitidos, Fletcher es la mejor opción.

En cuanto a la detección de errores, los dos algoritmos anduvieron muy bien, con tasas cercanas al 100 % en todas las pruebas. Esto nos dice que son igual de confiables para identificar si algo se corrompió durante la transmisión. También vimos que el tamaño del mensaje no influye prácticamente en esto: tanto con mensajes cortos como largos, el rendimiento se mantuvo estable.

Donde Hamming realmente brilla es en la corrección. Poder arreglar errores de un solo bit por bloque es una ventaja enorme. Pero ojo, nuestras pruebas también mostraron que esta capacidad se resiente cuando la tasa de error del canal es alta. Si varios bits de un mismo bloque se dañan, Hamming ya no puede garantizar una corrección correcta, y por eso vemos que la tasa de corrección baja en esos escenarios.

A la luz de los resultados, podemos afirmar que Fletcher constituye una solución adecuada cuando el único requisito es la detección de errores y se dispone de la posibilidad de retransmitir la información en caso de ser necesario. Su menor overhead lo convierte en una opción eficiente para estos escenarios. Por su parte, el Código de Hamming justifica su mayor redundancia cuando la retransmisión no es viable o se desea evitar. Su capacidad de corregir errores simples en el momento de la recepción representa una ventaja significativa en entornos donde el costo de reenviar datos es elevado o simplemente no está contemplado.

Estos resultados evidencian el equilibrio inherente entre la cantidad de redundancia agregada y el nivel de confiabilidad alcanzado. Incrementar la información de control en la trama mejora la resiliencia frente al ruido del canal, pero implica un mayor volumen de datos transmitidos. La elección entre ambos algoritmos dependerá, en última instancia, de las restricciones y prioridades de cada aplicación en particular.

Comentario Grupal

A lo largo del desarrollo de este laboratorio, pudimos comprender de manera práctica el funcionamiento de los mecanismos de detección y corrección de errores empleados en redes de comunicación. La implementación de ambos algoritmos permitió contrastar directamente las implicancias de agregar redundancia con fines exclusivos de detección frente a su uso para corrección. Asimismo, los resultados obtenidos reforzaron la importancia de seleccionar el algoritmo más apropiado en función de las características del canal y los requisitos particulares de cada aplicación.

Conclusiones

En este trabajo logramos implementar una arquitectura por capas funcional que permitió la comunicación entre un emisor en Python y un receptor en Node.js usando sockets TCP, incorporando tanto mecanismos de detección como de corrección de errores.

Los resultados mostraron que Fletcher Checksum es una opción eficiente cuando se trata solo de detectar errores: genera menos overhead que Hamming y mantuvo una tasa de detección muy alta en todas las pruebas que realizamos.

Por su parte, el Código de Hamming cumplió con su objetivo de detectar y corregir errores de un solo bit por bloque. Sin embargo, también vimos que su rendimiento cae cuando aumenta la probabilidad de que varios bits de un mismo bloque se dañen, algo que era esperable dado su diseño.

En conjunto, las pruebas confirmaron lo que ya anticipábamos: hay un equilibrio entre la redundancia que agregamos y la capacidad de recuperación que obtenemos. Cuanta más información de control incluimos en la trama, mejor podemos hacer frente al ruido del canal, pero a costa de transmitir más datos. La elección entre uno u otro algoritmo dependerá, en definitiva, de lo que priorice cada situación.

Referencias

- Kurose, J. F., & Ross, K. W. (2021). *Computer Networking: A Top-Down Approach* (8th ed.). Pearson.
- Stallings, W. (2020). *Data and Computer Communications* (11th ed.). Pearson.
- Universidad del Valle de Guatemala. (2026). *Laboratorio 2: Esquemas de detección y corrección de errores*. Facultad de Ingeniería.